

系统架构设计师

第1章 绪论

授课：王建平

目录

- 1 系统架构概述
- 2 系统架构设计师概述
- 3 如何成为一名好的系统架构设计师

目录

1

系统架构概述

2

系统架构设计师概述

3

如何成为一名好的系统架构设计师

系统架构的定义

1、概念 (★)

架构是体现在组件中的一个系统的基本组织、它们彼此的关系与环境的关系及指导它的设计和发展的原则。系统是组织起来完成**某一特定功能或一组功能的组件集**。系统这个术语包括了单独的应用程序、传统意义上的系统、子系统、系统之系统、产品线、整个企业及感兴趣的其他集合。

■ **系统**用于完成其环境中的一个或多个任务。

■ **环境或者上下文**决定了对这个系统的开发、运作、政策以及会对系统造成其他影响的环境和设置。

■ **任务**是由一个或者多个利益相关者通过系统达到一些目标的系统的一个用途或操作。

2、系统架构 (System Architecture) 是系统的一种**整体的高层次**的结构表示，是系统的骨架和根基，支撑和链接各个部分，包括**组件、连接件、约束规范**以及指导这些内容设计与演化的原理，它是刻画系统整体抽象结构的一种手段。系统架构设计的目的是对需要开发的系统进行一系列相关的抽象，用于指导系统各个方面的设计与实现，架构设计在系统开发过程中起着关键性作用，**架构设计的优劣决定了系统的健壮性和生命周期的长短**。

3、架构设计的作用主要包括以下几点： (★)

- 解决相对复杂的需求分析问题；
- 解决**非功能属性**在系统占据重要位置的设计问题；
- 解决生命周期长、扩展性需求高的系统整体结构问题；
- 解决系统基于组件需要的集成问题；
- 解决业务流程再造难的问题。

----系统架构设计是成熟系统开发过程中的一个重要环节，是连接**用户需求和系统进一步设计与实现**的桥梁，也是系统早期阶段质量保证的关键步骤。

软件架构发展阶段（★）

阶段	要 点
基础研究阶段	模块化开发方法 ---模块化开发方法涉及的主要问题是模块设计的规则，即系统如何分解成模块。而每一模块都可独立开发与测试，最后再组装成一个完整软件。将系统分解成模块时，应该遵循以下规则： （1）最高模块内聚。（2）最低耦合。（3）模块大小适度。（4）模块调用链的深度（嵌套层次）不可过多。（5）接口简单、精炼（扇入扇出数不宜太大），具有信息隐蔽能力。（6）尽可能地复用已有模块。
概念体系和核心技术形成阶段	成果：软件组件化技术。组件具有可组装性和可插拔性。每个组件的运行仅依赖于平台或者容器，组件与组件之间不存在直接的耦合关系。同时，组件和组件之间又并非绝对独立。组件经过组装后可以与其他组件进行业务上的交互。组件化开发并不等同于模块化开发。 模块化开发只是在逻辑上做了切分，物理上（代码）通常并没有真正意义上的隔离。 组件化也不等同于应用集成，应用集成是将一些基于不同平台或不同方案的应用软件有机地集成到一个无缝的、并列的、易于访问的单一系统中，以建立一个统一的综合应用。 组件化比模块化更独立，但比应用集成结合得更加紧密。

软件架构发展阶段

阶段	要 点
理论体系完善与发展阶段	软件工程领域 ：主要包括：软件架构描述与表示、软件架构分析、设计与测试；软件架构发现、演化与重用；于软件架构开发方法；软件架构风格；动态软件架构等。
普及应用阶段	软件架构 是软件生命周期中的重要产物，它影响软件开发的各个阶段。

软件架构的常用分类及建模方法（★★★）

比较典型的架构模型包括分层架构、事件驱动架构、微核架构、微服务架构和云架构等五类。

1、分层架构：将软件分成若干个水平层，每一层都有清晰的角色和分工，不需要知道其他层的细节。层与层之间通过接口进行通信。



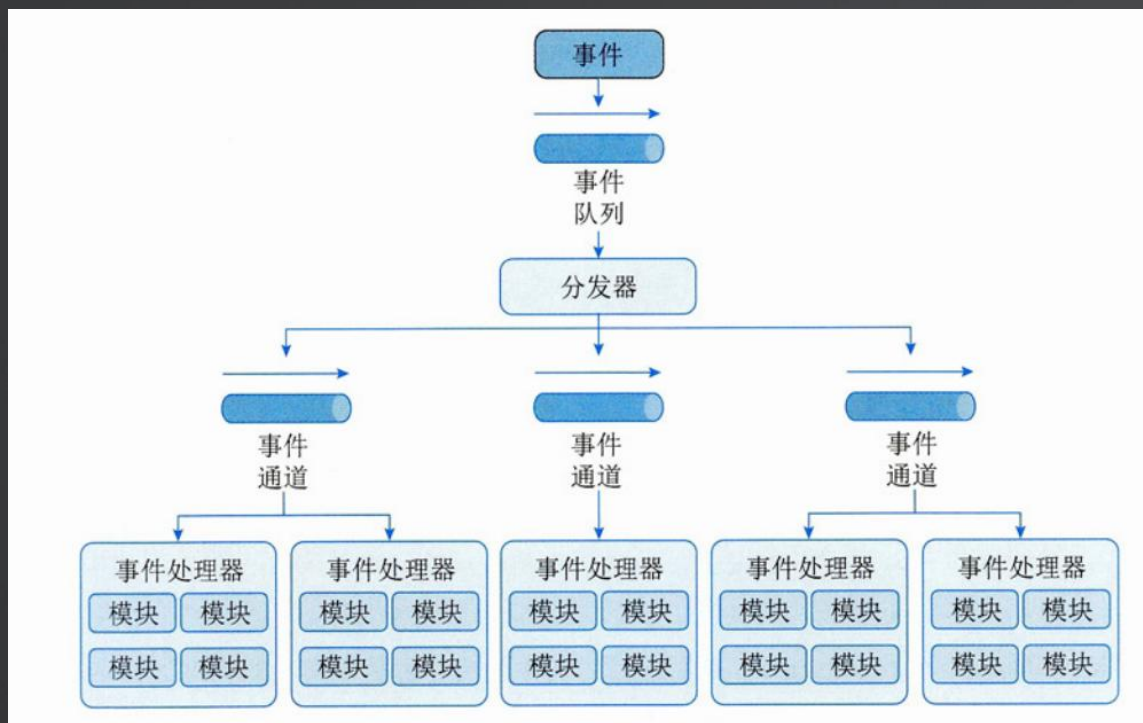
- ①表现层 (Presentation Layer)：用户界面，负责视觉和用户互动；
- ②业务层 (Business Layer)：实现业务逻辑；
- ③持久层 (Persistence Layer)：提供数据，SQL语句就放在这一层；
- ④数据库 (Database Layer)：保存数据。

有的项目在逻辑层和持久层之间加了一个服务层 (Service)，提供不同业务逻辑需要的一些通用接口。用户的请求将依次通过这四层的处理，不能跳过其中任何一层。

软件架构的常用分类及建模方法 (★★★)

2) 事件驱动架构

事件驱动架构(EDA)是通过**事件**进行通信的软件架构，是一种流行的**分布式异步**架构模式，适用于松散耦合系统。分成四个部分：

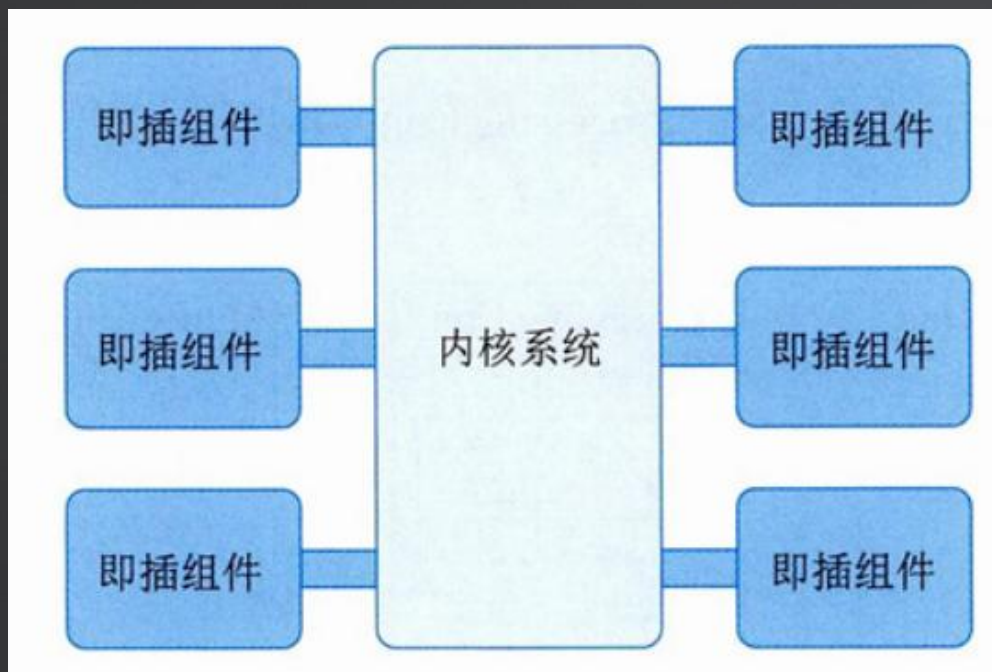


- ①**事件队列 (Event Queue)**：接收事件的入口；
 - ②**分发器 (Event Mediator)**：将不同的事件分发到不同的业务逻辑单元；
 - ③**事件通道 (Event Channel)**：分发器与处理器之间的联系渠道；
 - ④**事件处理器 (Event Processor)**：实现业务逻辑，处理完成后会发出事件，触发下一步操作。
- 对于简单的项目，事件队列、分发器和事件通道可以合为一体，整个软件就分成事件代理和事件处理器两部分。

软件架构的常用分类及建模方法（★★★）

3) 微核架构

微核架构（Microkernel Architecture）又称为**插件架构**（Plug-in Architecture），是指**软件的内核相对较小**，主要**功能和业务逻辑都通过插件实现**。内核（Core）通常只包含系统运行的最小功能。插件则是互相独立的，插件之间的通信应该减少到最低，避免出现互相依赖的问题。

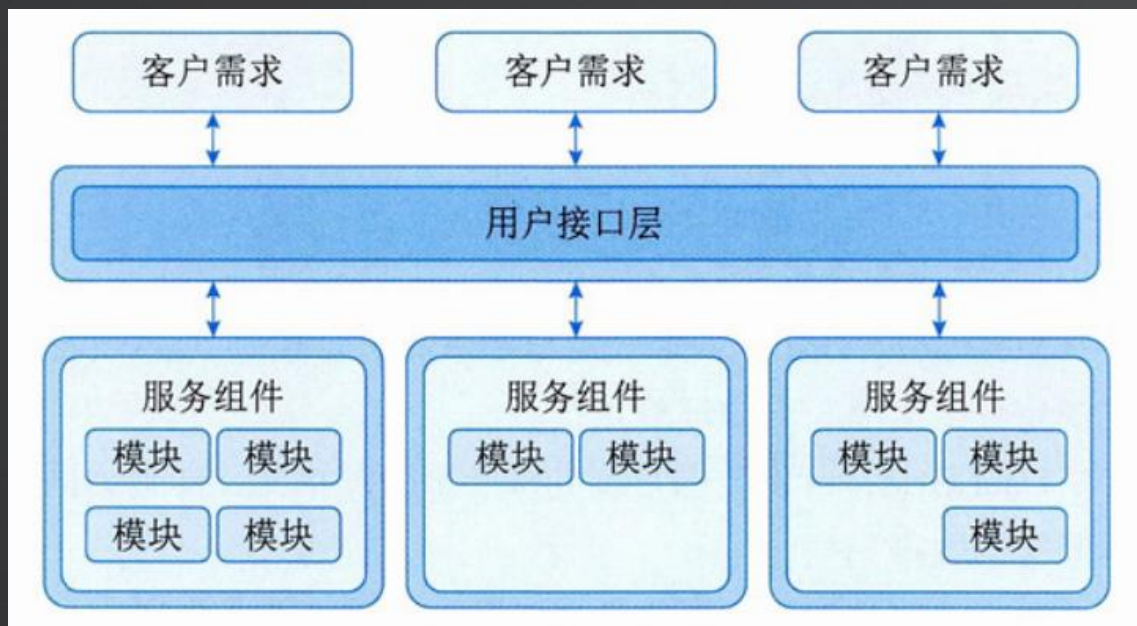


- **核心系统(内核)**: 负责和具体业务功能无关的通用功能(系统运行的最小功能), 例如模块加载、模块间通信等。
- **插件模块**: 负责实现具体的业务逻辑, 插件是互相独立的, 插件之间的通信应该减少到最低, 避免出现互相依赖的问题。

软件架构的常用分类及建模方法（★★★）

4）微服务架构

微服务架构（Microservices Architecture）是**服务导向架构**（Service-Oriented Architecture, SOA）的升级。每一个服务就是一个独立的部署单元（Separately Deployed Unit）。这些单元都是分布式的，互相解耦，通过远程通信协议（比如REST, SOAP）联系



微服务架构分成三种实现模式：

- **RESTful API模式**：服务通过API提供，云服务就属于这一类；
- **RESTful应用模式**：服务通过传统的网络协议或者应用协议提供，背后通常是一个多功能的应用程序，常见于企业内部；
- **集中消息模式**：采用消息代理（Message Broker）可以实现消息队列、负载均衡、统一日志和异常处理，缺点是会出现单点失败，消息代理可能要做成集群。

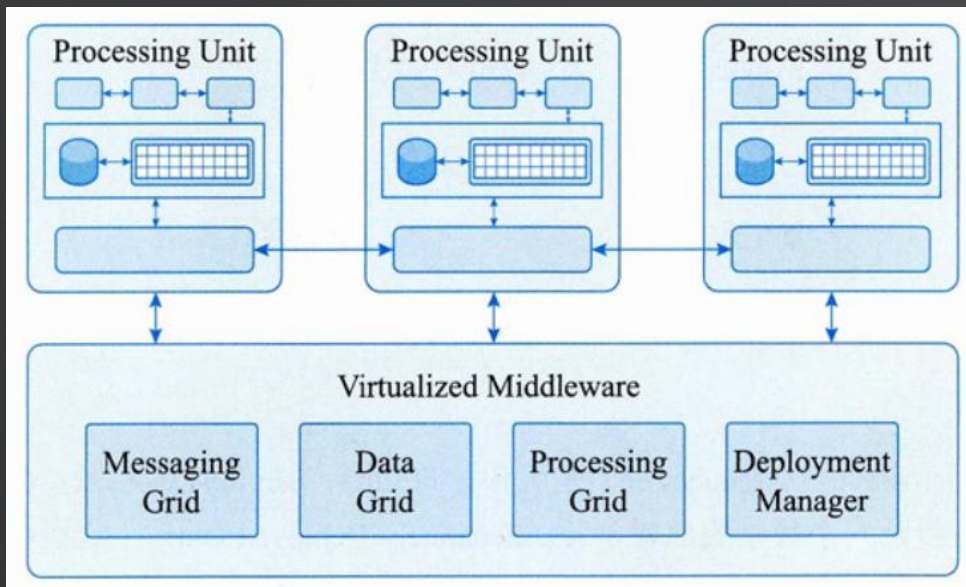
软件架构的常用分类及建模方法 (★★★)

5) 云架构

云架构 (Cloud Architecture) 主要解决扩展性和并发的问题，是最容易扩展的架构。它的高扩展性体现在将数据都复制到内存中，变成可复制的内存数据单元，然后将业务处理能力封装成一个个处理单元

(Processing Unit)。若访问量增加，就新建处理单元；若访问量减少，就关闭处理单元。

由于没有中央数据库，所以扩展性的最大瓶颈消失了。由于每个处理单元的数据都在内存里，需要进行数据持久化。云架构主要分成两部分：处理单元 (Processing Unit) 和虚拟中间件 (Virtualized Middleware)。



(1) 处理单元：实现业务逻辑。

(2) 虚拟中间件：负责通信、保持会话控制 (sessions)、数据复制、分布式处理和处理单元的部署。

虚拟中间件又包含四个组件：

- 消息中间件 (Messaging Grid)：管理用户请求和会话控制 (sessions)，当一个请求进来以后，它决定分配给哪一个处理单元。
- 数据中间件 (Data Grid)：将数据复制到每一个处理单元，即数据同步。保证某个处理单元都得到同样的数据。
- 处理中间件 (Processing Grid)：可选，如果一个请求涉及不同类型的处理单元，该中间件负责协调处理单元。
- 部署中间件 (Deployment Manager)：负责处理单元的启动和关闭，监控负载和响应时间，当负载增加，就新启动处理单元，负载减少，就关闭处理单元。

典型真题

下面不属于微服务架构分成三种实现模式（ ）。

A.RESTful API模式 B.RESTful应用模式 C.集中消息模式 D.分布式消息模式.

【答案】D

软件架构的模型（★★）

系统架构的常用建模方法，根据建模的侧重点的不同，可以将分成4种：**结构模型**、**框架模型**、**动态模型**和**过程模型**。

- 1) 结构模型：这是一个**最直观、最普遍**的建模方法。此方法以架构的**构件、连接件和其他概念**来刻画结构。并力图通过结构来反映系统的重要语义内容，包括系统的配置、约束、隐含的假设条件、风格和性质。研究结构模型的核心是**架构描述语言**。
- 2) 框架模型：框架模型与结构模型类似，但它**不太侧重描述结构的细节**，而更侧重**整体的结构**。框架模型主要以一些特殊的问题为目标建立只针对和适应问题的结构。
- 3) 动态模型：动态模型是对结构或框架模型的补充，主要研究系统的**“大颗粒”行为的性质**。例如，描述系统的重新配置或演化。这里的动态可以是指系统总体结构的配置、建立或拆除通信或计算的过程，这类系统模型常是激励型的。
- 4) 过程模型：过程模型是研究构造系统的**步骤和过程**，其结构是遵循某些过程脚本的结果。

这4种模型并不是完全独立的，通过有机的结合才可形成一个完整的模型来刻画软件架构，也将能更加准确、全面地反映软件架构。软件架构可从不同角度来描述用户所关心架构的特征。

典型真题

系统架构的常用建模方法，根据建模的侧重点的不同，可以将软件架构的模型分成4种：结构模型、框架模型、动态模型和过程模型。（1）是对结构或框架模型的补充，主要研究系统的“**大颗粒**”**行为的性质**。（2）是研究构造系统的**步骤和过程**，其结构是遵循某些过程脚本的结果。

- (1) A.结构模型 B.框架模型 C.动态模型 D.过程模型
(2) A.结构模型 B.框架模型 C.动态模型 D.过程模型

【答案】CD

软件架构应用场景及未来发展（★）

1、软件架构风格在实践中已被反复使用，不同的架构风格具有各自的优缺点和应用场景，比如

架构风格	适用场景
管道-过滤器风格	用于将系统分成若干独立的步骤
主程序 / 子系统和面向对象的架构风格	用于对组件内部进行设计
虚拟机风格	用于构造解释器或专家系统
C/S 和B/S 风格	适合于数据和处理分布在一定范围，通过网络连接构成系统
平台 / 插件风格	用于具有插件扩展功能的应用程序
MVC 风格	用于用户交互程序的设计
SOA 风格	用在企业集成等方面
C2 风格	用于GUI软件开发，用以构建灵活和可扩展的应用系统等

而对于现代大型软件，很少使用单一的架构风格进行设计与开发，而是混合多种风格，从不同视角描述大型软件系统的能力，并可保证软件系统的可靠性、可扩展性、可维护性等非功能属性的正确描述。

2、软件架构发展未来

架构发展的主线可以归纳为模块化编程 / 面向对象编程、构件技术、面向服务开发技术和云技术。

典型真题

() 架构风格用于GUI软件开发，用以构建灵活和可扩展的应用系统等
A.管道-过滤器风格 B.MVC 风格 C.SOA 风格 D.C2 风格

【答案】 D

() 用于对组件内部进行设计。
A.管道-过滤器风格 B.MVC 风格 C.SOA 风格 D.面向对象风格

【答案】 D

本章重点回顾

- 1、系统架构概念
- 2、系统常见架构及建模方法
- 3、系统架构应用场景

THANKS